

Java – Beautifully Broken

Java 27 quietly changes your JVM's defaults, and Maven starts saying no

Issue #6

Sat, 10 Oct 2026

 ~36 min read

Level 3/5



Today's two stories

1. **Java 27 changes three defaults you probably never set yourself.** G1 is now the garbage collector everywhere, including small containers that used to get Serial GC. Every object's header shrinks from 12 bytes to 8. And G1 no longer gives heap back to the OS the way it used to. Together they change how much memory your service uses after an upgrade, sometimes for the better and sometimes not.
2. **Maven is in the middle of its biggest change in 16 years.** Maven 3.10.0, now the recommended release, turns old warnings into build failures, changes classpath order and scopes your repository passwords. Maven 4.0.0-rc-7, the last planned release candidate, is the preview of what comes next: a new POM model, "subprojects" and a POM split in two. Here's what each means for your build.
3. **On the radar:** 6 more things that happened, listed at the end.

STORY 1

Java 27 changed your JVM's defaults. Here's what that does to memory

JDK 27 · HOTSPOT · JEP 523 AND JEP 534 · RELEASED 15 SEP 2026 · ~15 MIN READ

Here's a question most Java teams can't answer: which garbage collector is your service running in production right now? If you never passed a flag, the JVM chose one for you at startup, and if your Kubernetes pod is limited to one CPU, it very likely chose the collector designed for embedded devices. Java 27 ends that special case. It also shrinks every object in your heap and changes when the heap gives memory back. None of this needs a code change, and that's exactly why it's worth understanding before you upgrade.

WHAT HAPPENED

JDK 27 was released on 15 September 2026 (full version string `27+35`).¹ It contains nine JEPs. Most of the attention went to previews and post-quantum TLS, but two of the finished features change how every application behaves when it starts:^{2,3}

- **JEP 523, "Make G1 the Default Garbage Collector in All Environments".** Oracle's release notes describe it in one line: "Make the Garbage-First (G1) garbage collector the default collector in all environments".¹ It had been targeted to JDK 27 in May.⁴
- **JEP 534, "Compact Object Headers by Default".** In the release notes' words, it will "make compact object headers the default object header layout in the HotSpot JVM".¹

A third change has no JEP, but it matters just as much for memory. Under JDK-8238686, the default values of `-XX:MinHeapFreeRatio` and `-XX:MaxHeapFreeRatio` changed "from 40 and 70 to 0 and 100".¹ That sounds like trivia, but it changes when G1 hands memory back to the operating system, and at least one open-source project has already had to work around it.⁵

These are the defaults you'll inherit whenever you move to Java 27 or any later release, including the next LTS. So even if you're on 17 or 21 today, this is a preview of your future production memory profile.

THE BACKGROUND YOU NEED

What a garbage collector does, and why there's a choice

Java frees memory for you. A **garbage collector** *≈ the part of the JVM that finds objects your program can no longer reach and reclaims their memory* (GC) runs periodically to do this. Most collectors briefly stop your application while they work, and that interruption is called a **pause** *≈ a period during which the GC halts application threads; it shows up as latency spikes*.

HotSpot, the JVM in OpenJDK, ships several collectors with different trade-offs:

- **Serial GC** does all its work on a single thread while your application is stopped. It has almost no bookkeeping overhead, which makes it efficient on tiny machines, but its pauses grow with the size of the heap.
- **G1** (short for **Garbage-First** *≈ HotSpot's general-purpose collector, which splits the heap into equal-sized regions and collects the regions with the most garbage first*) is designed to keep pauses short and predictable. It does part of its work **concurrently** *≈ at the same time as your application runs, on separate GC threads* and cleans up the old part of the heap a few regions at a time, rather than all at once.
- **Parallel GC** and **ZGC** cover throughput-first and ultra-low-latency use cases.

How the JVM picked a collector before Java 27

If you don't choose a collector on the command line, the JVM chooses one at startup. This is called **ergonomics** *≈ the JVM's automatic selection of settings such as the GC and heap size, based on the machine it starts on*. G1 became the default "for server environments" in JDK 9 through JEP 248.⁶ But there was an exception. Nicolai Parlog's heads-up for JDK 27 explains it: the JVM "would select the latter [Serial] if only a single CPU or less than 1792 MB of physical memory was detected."⁷

That exception was written for small physical machines. Then everything moved into containers. The JVM is **container-aware** *≈ able to read the CPU and memory limits Linux applies to a container, instead of seeing the whole host*, and it works out its CPU count from the container's quota. Microsoft's Kubernetes guide spells out the consequence: "any value up to `1000m` millicores are identified as a single processor machine."⁸ In other words, a pod with a CPU limit of 1 or less looked like a single-CPU machine, and got Serial GC.

We checked this on JDK 21 by telling the JVM it had one processor:

```
$ java -XX:+PrintFlagsFinal -version | grep 'UseSerialGC\|UseG1GC'

bool UseG1GC      = true      {product} {ergonomic}
bool UseSerialGC  = false     {product} {default}

$ java -XX:ActiveProcessorCount=1 -XX:+PrintFlagsFinal -version | grep ...

bool UseG1GC      = false     {product} {default}
bool UseSerialGC  = true      {product} {ergonomic}
```

The word `{ergonomic}` means the JVM made that choice, not you. As PVS-Studio's Egor Pishii put it in a quote picked up by The Register, "A chunk of containerized Java apps has been running on [Serial] all this time."³

Why the exception no longer made sense

Serial was kept for small environments because it was cheaper there. The JEP lists what changed. G1's **write barrier** *≈ a few extra instructions the JVM runs every time your code stores a reference into an object field, so the GC can track pointers between parts of the heap* used to be expensive. JDK 26's JEP 522 gave G1 a second **card table** *≈ a compact map that records which small areas of the heap have had references written into them recently*, so application threads and GC threads stop getting in each other's way. The JEP reports that write barriers shrank "from around 50 instructions to just 12" on x64, with throughput gains of 5–15% for applications that update reference fields heavily.⁹

With that in place, JEP 523 argues that "G1 is now competitive with Serial at all heap sizes", that its maximum throughput "is close to that of Serial", that its maximum latencies are better because it cleans the old generation incrementally, and that its native memory use is now comparable.⁶ Removing the special case also makes the JVM easier to reason about: one default everywhere.

How Java 27's defaults came about



Object headers, and Project Lilliput

Every Java object starts with a hidden **object header** *≈ metadata the JVM stores at the start of every object, before your fields*. On a typical 64-bit JVM it has two parts:¹⁰

- The **mark word** *≈ the first 64 bits of the header, holding the identity hash code, the GC age and the lock state* : a 31-bit identity hash code, 4 bits of GC "age", and 2 lock bits. The article on HappyCoders counts 27 of its 64 bits as unused.
- The **class word** *≈ the part of the header that points to the object's class metadata* : normally a 32-bit **compressed class pointer** *≈ a 32-bit offset into a reserved class-metadata area, used instead of a full 64-bit address to save space* .

That's 96 bits, or 12 bytes, before a single field of yours. For a small object such as a `Long` , the header is bigger than the data. **Project Lilliput** *≈ the OpenJDK project working to shrink object headers, named after the tiny people in Gulliver's Travels* has spent years trying to squeeze it. Its first result, compact object headers, arrived as an experiment in JDK 24 (JEP 450), became a supported but opt-in feature in JDK 25 (JEP 519), and is now on by default.¹⁰

HOW IT WORKS

Change 1: G1 is chosen everywhere

The rule is now simple. If you don't name a collector, "the JVM always selects G1, regardless of the number of processors and the Java heap size."⁶ Thomas Schatzl, who owns the JEP, summarised it more bluntly: "No exceptions, no Serial GC in any case by default."¹¹



Three things don't change. Serial GC isn't being removed. If you select a collector on the command line, your choice still wins. And machines with more than one CPU and at least 1792 MB were already getting G1.⁷ Parlog's heads-up summarises the trade-off for the small environments that are affected: compared with Serial, G1 has "similar native memory overhead, slightly lower throughput, and lower maximum latencies."⁷

To keep the old behaviour, add `-XX:+UseSerialGC` .¹² To make startup a bit cheaper for G1, JDK 27 also postpones most of the setup for G1's concurrent marking (JDK-8367993), which gave "a small reduction in VM creation time."¹²

Change 2: every object loses 4 bytes of header

A compact header merges the mark word and the class word into a single 64-bit value. The trick is to make the class pointer smaller. Class metadata is stored in an area of up to 4 GB, and JDK 27 lays it out in 1,024-byte blocks. A 22-bit block number, shifted left by 10 bits, rebuilds the full 32-bit offset. That frees 10 bits, which cover the remaining gap and leave room for new uses.¹⁰

The new 64-bit header, field by field

Class pointer · 22 bits

was a separate 32-bit word

Identity hash code · 31 bits

unchanged

Reserved for Valhalla · 4 bits

value objects

GC age · 4 bits

unchanged

Self-forwarded tag · 1 bit

new: helps the GC

Lock bits · 2 bits

unchanged

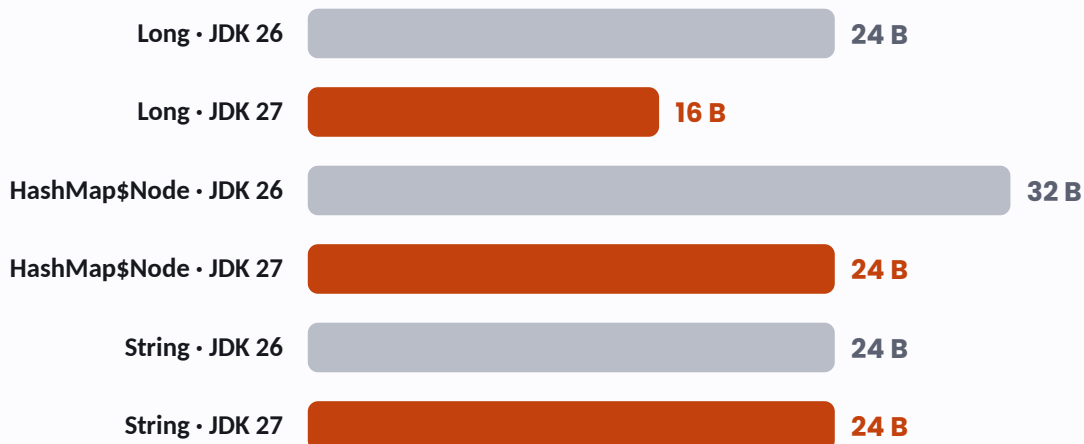
The **self-forwarded tag** *≈ a single header bit the GC sets when it couldn't move an object and the object therefore "forwards" to itself* exists because of how collectors move objects. While a GC copies an object, it normally overwrites the old header with a **forwarding pointer** *≈ the new address of an object that the GC has just moved, written into the old copy*. With a compact header, that would overwrite the class pointer, which the JVM needs in order to know how big the object is. So in the special case where an object can't be moved, the GC now sets one bit instead of writing a pointer to itself.¹⁰

One more detail explains why the savings vary. HotSpot rounds every object up to a multiple of 8 bytes, a rule called **object alignment** *≈ the JVM's practice of starting every object on an 8-byte boundary, which pads object sizes up to the next multiple of 8*. Saving 4 bytes only helps if it drops the object below one of those boundaries. Do the arithmetic and you can predict what happens (assuming the usual 4-byte object references):

- `Long` : 12-byte header + 8-byte value = 20, padded to **24**. Compact: 8 + 8 = **16**. One-third smaller.
- `String` : 12 + 4 (reference to its byte array) + 4 (`hash`) + 1 (`coder`) + 1 (`hashIsZero`) = 22, padded to **24**. Compact: 18, padded to **24**. No change.

We confirmed the "before" sizes on JDK 21 with a heap histogram: `Long` and `String` both took 24 bytes per instance. Ankur, who writes at ankurm.com and measured a Spring Boot 4.1.1 application on JDK 26 and JDK 27, saw exactly the "after" numbers: `Long` fell from 24 to 16 bytes, `HashMap$Node` from 32 to 24, and `String` stayed at 24.¹³

Bytes per instance: JDK 26 default vs JDK 27 default



Source: Ankur's class histograms of a Spring Boot 4.1.1 app on Temurin 26.0.2.1 and 27+35 (ankurm.com, 24 Sep 2026). Our own JDK 21 histogram matched the JDK 26 column for Long and String.

How much does that add up to? The figures from JEP 519, quoted in Oracle's JDK 27 performance round-up, were "22% lower heap usage and 8% lower CPU usage in one SPECjbb2015 configuration" and "15% fewer collections with both G1 and Parallel GC in another".¹² **SPECjbb2015** *≈ an industry-standard Java server benchmark that simulates a supermarket company's IT system* is a benchmark, not your app, and the same article warns that "an application dominated by large primitive arrays will benefit less than one dominated by small objects."¹² Ankur's real Spring Boot test, which held 2 million small records in a `HashMap`, saw its **live set** *≈ the memory occupied by objects still in use after a full garbage collection* fall from 320 MB to 273 MB, about 15%.¹³

Change 3: the heap stops shrinking on its own

G1 can grow and shrink the heap while your application runs. For years, two flags guided the shrinking after a full collection: `MinHeapFreeRatio` (grow if less than 40% is free) and `MaxHeapFreeRatio` (shrink if more than 70% is free). Schatzl explains that these "(arbitrary)" percentages and G1's newer heuristic, which sizes the heap based on how much CPU time the GC is using, would "fight each other", causing pointless resizing. So in JDK 27, G1 "no longer resizes the heap after Full GC based on" them, and their defaults became 0 and 100.¹¹ If you set either flag yourself, the old behaviour still applies.¹

The side effect is that a heap that has grown may stay large. The maintainers of `tunl`, an open-source tool, found this the hard way. Their PR describes a user's instance with all 512 MB of `-Xmx` committed for just 46 MB of live data, and a forced full GC still left 512 MB. They also saw G1's **periodic GC** *≈ an optional timer-driven collection G1 runs when the application is idle, intended to return unused memory* make the heap grow while idle, from 185 MB to 497 MB in two minutes in one test. Their fix, merged on 28 September, sets `-XX:MaxHeapFreeRatio=70` explicitly and adds `-XX:-G1PeriodicGCInvokesConcurrent`; in an eight-minute test, the committed heap dropped from 173 MB to 120 MB.⁵

Smaller things that will show up in your logs

Two related housekeeping changes may surprise your scripts. `-XX:InitiatingHeapOccupancyPercent`, the threshold for starting G1's concurrent marking cycle, has been renamed `-XX:G1IHOP`; the old name still works but is deprecated. And `UseCompressedClassPointers` is now obsolete: passing it prints "Ignoring option `UseCompressedClassPointers`; support was removed in 27.0".¹

WHAT IT MEANS FOR YOU

If you size containers tightly, you may now be able to make them smaller. Ankur's container sweep is the clearest real-world picture so far. With the heap set to 75% of the container's memory, the smallest container that could load his data set was 448 MB on JDK 26 and 400 MB on JDK 27. With the JVM's default heap (25% of the container), it was 1,536 MB on JDK 26 and 1,152 MB on JDK 27.¹³ Most of that came from compact headers: with them switched off on JDK 27, the minimums went back up to 432 MB and 1,280 MB.

If your pods have one CPU, your GC just changed. In Ankur's one-CPU, 1 GB container, JDK 26 picked Serial and JDK 27 picked G1. The resident memory was 529 MB on JDK 26 and 436 MB on JDK 27; forcing Serial on JDK 27 gave 482 MB. One detail is worth knowing: in that run, G1's own data structures took about 43 MB of native memory against Serial's 1.7 MB, according to the JVM's [Native Memory Tracking](#) *≈ a built-in JVM feature that reports memory used outside the Java heap, by category*. Compact headers more than paid for it in this application, but on a different workload the balance could tip the other way.¹³

One CPU, 1 GB container, 512 MB heap: resident memory (RSS)



Medians of 3 runs from Ankur's Spring Boot 4.1.1 measurements (ankurm.com, 24 Sep 2026). One workload on one machine: measure your own.

A practical upgrade checklist:

1. Find out what you're running today. Start a pod with `-Xlog:gc` and look for the line that says `Using Serial` or `Using G1`.
2. If you were on Serial without knowing it, run a load test on JDK 27's default G1 and on `-XX:+UseSerialGC`. Parlog's own advice is to "benchmark the application against different GCs."⁷
3. Watch committed heap over a long, idle period, not just peak usage. If it no longer comes down, consider setting `MaxHeapFreeRatio` again, as tunl did.
4. Check your agents. Ron Veen's write-up on JEP 534 warns that code using `sun.misc.Unsafe` to read headers by offset, some JVMTI agents, profilers and serialization frameworks "may need review."¹⁴ Keep `-XX:-UseCompactObjectHeaders` handy for an A/B comparison, which is how Oracle's round-up suggests using it.¹²

In the real world

Compact headers aren't new to production. The Register reports that Amazon already runs hundreds of production services with them, and that SAP made them the default in its own OpenJDK build, SapMachine.³ Veen adds that Amazon backported the feature to JDK 17 and 21 for its own fleet.¹⁴

Watch out for

- **The opt-out is temporary.** Oracle's release notes say the `-XX:-UseCompactObjectHeaders` flag "is planned for deprecation and removal in a future release."¹
- **G1 isn't best everywhere.** Schatzl himself says G1 is "certainly the best to start off with," but "not the best everywhere."¹¹ The JEP admits that "some applications in constrained environments will still perform best with Serial."⁶
- **Big arrays don't shrink.** If most of your heap is `byte[]` buffers or large arrays, 4 bytes per object barely registers.
- **Your heap-sizing flags may change meaning.** If you relied on the default free ratios to return memory to the OS, that no longer happens after a full GC unless you set them yourself.

WHAT HAPPENS NEXT

Lilliput isn't finished. The HappyCoders article describes a draft for 4-byte headers, which would move the identity hash code out of the header and shrink the class pointer to 19 bits, enough for "around 500,000 classes". That draft has no target release yet.¹⁰ Meanwhile Schatzl says the G1 team's "focus remains on Automatic Heap Sizing", which would let G1 respond to memory pressure on the machine instead of following fixed ratios.¹¹ In other words, the JVM is moving toward needing fewer flags, and Java 27 is a big step on that road.

Words from this story

Garbage collector (GC): reclaims memory of unreachable objects.

Pause: time the GC halts your threads.

Garbage-First (G1): region-based, low-pause collector.

Concurrently: alongside your app, on GC threads.

Ergonomics: the JVM's automatic choice of settings.

Container-aware: reads the container's limits.

Write barrier: extra code run on each reference store.

Card table: map of recently written heap areas.

Object header: hidden metadata before your fields.

Mark word: header bits for hash, GC age and locks.

Class word: header part pointing to class metadata.

Compressed class pointer: 32-bit class offset.

Project Lilliput: OpenJDK's header-shrinking project.

Self-forwarded tag: bit set when an object can't move.

Forwarding pointer: a moved object's new address.

Object alignment: sizes rounded up to 8 bytes.

SPECjbb2015: a standard Java server benchmark.

Live set: memory still in use after a full GC.

Periodic GC: G1's timer-driven idle collection.

Native Memory Tracking: reports non-heap JVM memory.

STORY 2 Maven 3.10 gets strict, and Maven 4 is one step from GA

BUILD TOOLS · APACHE MAVEN · 3.10.0 RELEASED 27 SEP 2026 · 4.0.0-RC-7 RELEASED 24 SEP 2026 · ~14 MIN READ

Maven is the part of the Java world that never seems to change. The `pom.xml` you write today looks almost exactly like the one you wrote in 2012, and that's by design. But in the space of four days at the end of September, the Maven team shipped two releases that between them start to change that. One is already the version the Maven website tells everyone to use, and it will break some builds that worked last week. The other is the last stop before Maven 4.

WHAT HAPPENED

Two releases landed within days of each other.

Maven 4.0.0-rc-7 came out on 24 September.¹⁵ Its release notes say it "is expected to be the last before the Maven 4.0.0 GA release, which we aim to publish in the coming weeks, pending community feedback."¹⁶ InfoQ's roundup described it as fixing numerous bugs from rc-6, along with performance work in the **reactor** *≈ the part of Maven that works out the build order of the projects in a multi-project build*.¹⁷

Maven 3.10.0 followed on 27 September, according to Maven's release history; the GitHub release was published on 1 October.^{15,18} Maven's download page now says that 3.10.0 "is the latest release. It is the recommended version for all users", and that 3.9.16 "is still maintained by the Apache Maven team, but users are encouraged to upgrade to 3.10.0."¹⁹ The same page is just as clear about Maven 4: it "is **NOT** safe for production use."

So the release you'll actually adopt this autumn is 3.10, and Maven 4 is what to start testing.

 Maven 3.10.0	VS	 Maven 4.0.0-rc-7
• GA, recommended for all users • Runs on Java 8+ • Resolver 2, strict validation • POM model 4.0.0 only		• Release candidate, not for production • Runs on Java 17+ • Resolver 2 behind a new Maven API • Build POM 4.1.0, consumer POM 4.0.0

THE BACKGROUND YOU NEED

What Maven actually does

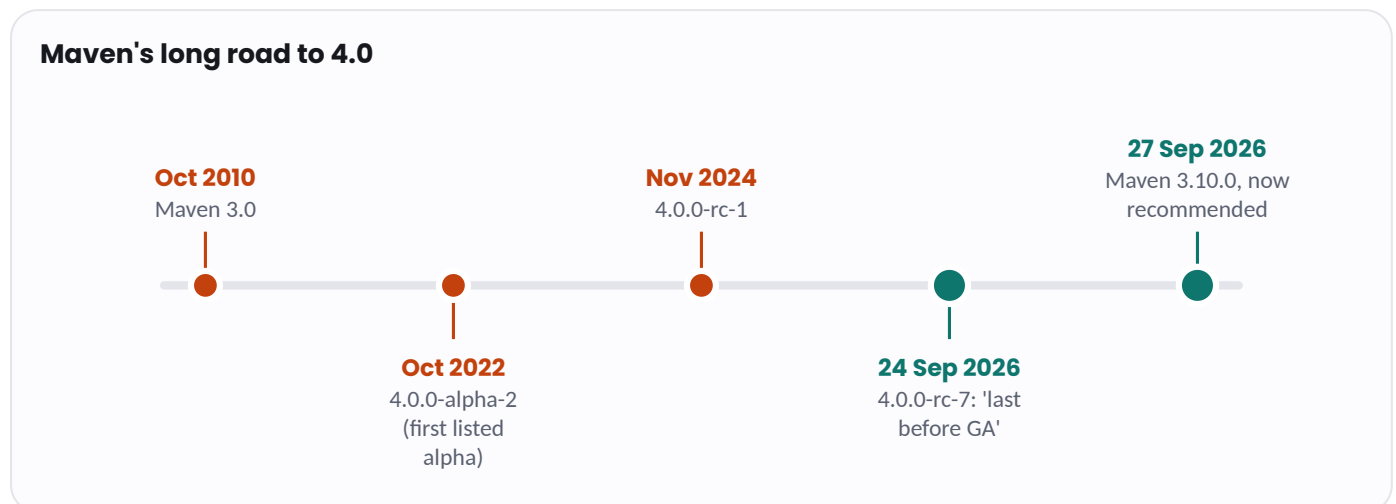
Maven builds Java projects from a **POM** *≈ Project Object Model: the pom.xml file that describes a project's coordinates, dependencies, plugins and build settings*. Its key job is managing **dependencies** *≈ the libraries your project needs, identified by groupId, artifactId and version*, including **transitive dependencies** *≈ the libraries that your libraries depend on, which Maven pulls in automatically*. The part of Maven that downloads and resolves them is the **Maven Resolver** *≈ the library inside Maven that resolves dependency graphs and downloads artifacts from repositories (formerly called Aether)*.

Projects often share settings through a parent POM, and many import a **BOM** *≈ Bill of Materials: a POM whose only job is to pin the versions of a set of related libraries, such as Spring Boot's*. Larger codebases have many projects in one **multi-module build** *≈ a parent POM that lists child projects, which Maven builds together in dependency order*.

Why Maven has barely changed since 2010

Maven 3.0 was released on 8 October 2010.¹⁵ Almost everything since has been a 3.x release, and the POM format still uses model version 4.0.0. The reason is that the POM plays two roles. It's the file you build from, and it's also the file that's published to **Maven Central** *≈ the main public repository from which Java build tools download open-source libraries* alongside every library. Gradle, sbt, IDEs and older Maven versions all read those published POMs. Change the format and you break every tool that consumes them. The "What's new in Maven 4" page quotes Hervé Boutemy: "With the Maven build schema preserved in amber, we can't evolve much."²⁰

Maven 4 has been in development for a long time. The first alpha listed in Maven's release history came out in October 2022, the first release candidate in November 2024, and there have now been seven release candidates in all.¹⁵



Why there's a 3.10 at all

Why release a new Maven 3 line when Maven 4 is so close? The 3.10.0 release notes answer directly: "The main driver for new generation was to align Maven 3 and 4 majors versions on same Resolver version. Hence, Maven 3.10 is now using Resolver 2.0 lineage, same one that is used by Maven 4+."²¹ In other words, 3.10 gives you Maven 4's dependency engine while keeping Maven 3's behaviour elsewhere. And unlike Maven 4, which needs Java 17, 3.10 still runs on Java 8.²¹

Maven's support policy makes the timing matter. The team "maintains the last version of the last two series of GA releases", currently 3.10.0 and 3.9.16. Maven 3.8.9 and earlier have reached end of life, and "New plugin releases will require Maven 3.9.0 or later."¹⁵

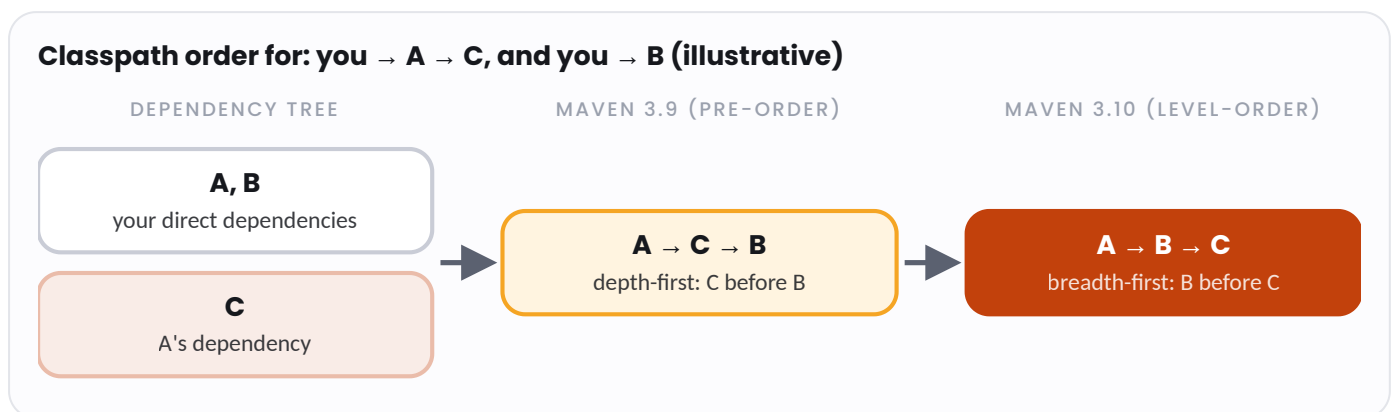
HOW IT WORKS

Maven 3.10: what changes in your build today

The release notes say that "at 'Maven level', changes are kept minimal". But four of those changes can turn a green build red.²¹

1. Warnings become errors. "Maven 3.10.0 raises the POM validation level: many WARNINGS from the 3.9 lineage (in fact, from the 3.1 lineage) will now FAIL the build. Things like duplicated dependency entries or duplicated plugin entries will not be tolerated anymore." Invalid characters in versions, malformed snapshot versions and reserved or malformed repository IDs also fail the build now. If your CI log has had the same POM warning in it for years, this is when it starts to matter.

2. The classpath order changes. "Dependency classpath ordering changed from 'pre-order' (depth-first) to 'level-order' (breadth-first)." Normally the order doesn't matter. It does when two JARs on the classpath *≈ the ordered list of JARs and directories the JVM searches for classes; the first match wins* contain the same class, because the first one wins. Here's an illustrative example: your project depends on **A** and **B**, and **A** depends on **C**.



If **B** and **C** both contain a class with the same name, Maven 3.10 now loads **B**'s copy. To go back to the old order, the release notes give `-Daether.system.dependencyVisitor=preOrder`.²¹ The better fix is to find and remove the duplicate class.

3. Your repository passwords are scoped. In `settings.xml`, each `<server>` entry holds credentials for a server id. In 3.10, "credentials of a `settings.xml` server are scoped to the origin (scheme, host and port) of the repositories and mirrors declared with the same server id, and the server id is matched exactly." This is a security hardening *≈ a change that closes off a way an attacker could abuse existing behaviour* measure: a password can no longer leak to a different host that happens to reuse the id. There's one catch. A repository declared in a profile that's activated with `<activation>` or `-P` "contributes no origin, so its credentials have to be declared explicitly", using a new `<repositoryOrigins>` list.²¹

```

<server>
  <id>internal</id>
  <username>u</username>
  <password>p</password>
  <repositoryOrigins>
    <repositoryOrigin>https://repo.example.org</repositoryOrigin>
  </repositoryOrigins>
</server>

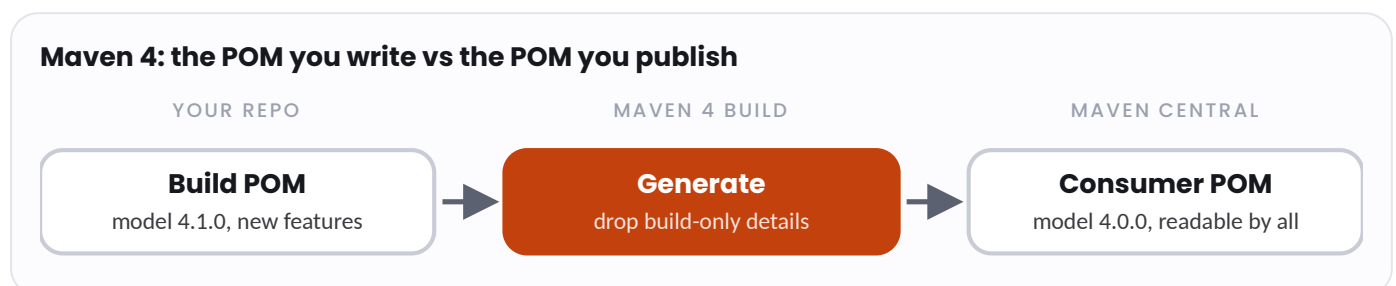
```

4. The Super POM changes. Every POM silently inherits from Maven's built-in **Super POM** *≈ the default POM built into Maven that every project inherits from, supplying default directories and plugin settings*. In 3.10 it "removed deprecated `release-profile`", "removed plugin management; each project should have its own plugin management", and sets `project.build.sourceEncoding` and `project.reporting.outputEncoding` to UTF-8 by default. It also turns on **reproducible builds** *≈ builds that produce byte-for-byte identical output from the same source, so anyone can verify a published artifact* by default, by supplying a default `project.build.outputTimestamp`.²¹

There are welcome additions too: user-wide extensions in `~/.m2/extensions.xml`, server `<aliases>` so one set of credentials can serve several ids, finer `-U` options (`-UA` for artifacts only, `-UM` for metadata only) and, from Resolver 2, version range filtering and user-defined **relocations** *≈ redirecting one artifact's coordinates to another, for example when a library is renamed*. If the stricter artifact validation blocks you, `-Dmaven.resolver.validation=mild` or `off` is an escape hatch, though the notes say "the coordinates should be fixed instead."²¹

Maven 4: two POMs instead of one

Maven 4's central idea is to separate the file you build from and the file you publish. The **build POM** *≈ the `pom.xml` in your source repository, which Maven 4 lets you write in the new 4.1.0 model* can use a new model version, 4.1.0. The **consumer POM** *≈ a cleaned-up POM that Maven 4 generates and publishes to the repository for other projects to read* stays at model 4.0.0, so Gradle, IDEs and older Maven versions can still read your published libraries. "It does not deploy the `pom.xml` used to build the project."²⁰



The consumer POM leaves out what only matters to your build, such as plugin configuration and POM properties, and keeps dependencies and repository information. Optionally, Maven can also **flatten** *≈ resolve a POM's parent and imported BOMs into one self-contained file* it, but that's off by default; you enable it with `maven.consumer.pom.flatten=true`.²⁰ One important reassurance from the same page: "Maven 4 will

continue to build your Maven 3's model version 4.0.0 project. There is no need to update your (build) POMs to 4.1.0 if you don't need to use the new features."

What the 4.1.0 model lets you write

Once you opt into model 4.1.0, a lot of long-standing irritations go away.²⁰

- **Modules become subprojects.** The new `<subprojects>` element replaces `<modules>`, which is deprecated but still works. If the parent has `pom` packaging and lists none, Maven discovers subdirectories that contain a `pom.xml`.
- **Children don't repeat the parent's coordinates.** With a relative path, the `<parent>` can omit its version, or even be written as `<parent/>`. This implements MNG-624, a request from 2005.
- **CI-friendly versions just work.** Maven 3.5 added partial support for `${revision}`, but you needed the Flatten Maven Plugin to make it work properly. In Maven 4 "You can now use **any variable** as a version", as long as it's defined at build time.
- **A real `bom` packaging type**, so a BOM no longer has to be a parent POM in disguise.
- **Multiple source directories** through a repeatable `<sources>` element, replacing the Build Helper plugin.
- **A `root` attribute and `${session.rootDirectory}`**, an official way to find the top of a multi-project build.

A child POM: model 4.0.0 vs model 4.1.0

Maven 3 (model 4.0.0)

```
<project xmlns=".../POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>com.acme</groupId>
    <artifactId>parent</artifactId>
    <version>1.4.0-SNAPSHOT</version>
  </parent>
  <artifactId>orders</artifactId>
</project>
```

Maven 4 (model 4.1.0)

```
<project xmlns=".../POM/4.1.0">
  <parent/> <!-- inferred from .. -->
  <artifactId>orders</artifactId>
</project>
```

Illustrative, based on the parent-inference rules in Maven's 'What's new in Maven 4' page. The `<parent/>` shorthand defaults to a relative path of `...`. Namespaces shortened.

Lifecycle, profiles and safety nets

Maven 4 also tidies up how builds run. Every phase gains `before:` and `after:` hooks, such as `before:integration-test`, with numbers in brackets to order plugins inside a phase. The old `pre-*` and `post-*` phases become aliases, and the docs say plainly: "Don't use them!" Asking for a profile that doesn't exist now fails the build, unless you mark it optional with `-P?name`. A new `--fail-on-severity WARN` option (`-fos`) fails the build on any warning. And `mvn -r` resumes a failed multi-project build from the subproject that broke.²⁰

The team also has a pointed piece of advice: "Do not use `mvn clean install` for your regular builds. Instead, use `mvn verify`!"²⁰

The migration tool

Maven 4 ships with `mvnup`, an upgrade assistant. The rc-7 release notes describe the two-step workflow:¹⁶

```
mvnup check    # detect potential problems
mvnup apply    # automatically apply recommended fixes
```

It can move POMs to model 4.1.0, flag deprecated features and migrate deprecated plugin configuration. For extensions it doesn't know about, the notes advise you to "contact the extension developers to confirm their plans for Maven 4 compatibility."¹⁶

WHAT IT MEANS FOR YOU

This week: try your builds on Maven 3.10. If you use the **Maven Wrapper** *≈ the mvnw script checked into a repository, which downloads and runs a pinned Maven version*, that's a one-line change to `.mvn/wrapper/maven-wrapper.properties`. Run a full build, and pay attention to anything that used to be a warning. Compare `mvn dependency:build-classpath` output before and after if you suspect duplicate classes.

Before a release: check your publishing pipeline. That's where the first serious 3.10 regression showed up (see the box below).

This quarter: run Maven 4 on a branch. You don't need to rewrite any POMs; Maven 4 builds model 4.0.0 projects. Run `mvnup check`, make sure your build machines have Java 17 for Maven itself (your code can still target older Java versions through the compiler's `release` setting or toolchains), and pin every plugin version, because "Maven 4 will issue a warning if you rely on default versions defined in Maven's Super POM."²⁰

In the real world

- **Jenkins is moving.** The Jenkins infrastructure team opened a tracking issue on 2 October to move its build agents and Docker images to Maven 3.10.0. Its packer images and packaging image have already been updated.²²
- **Publishing to Maven Central broke for some.** On 8 October a user reported that with Maven 3.10.0, Sonatype's `central-publishing-maven-plugin` 0.11.0 produced a staging bundle that Maven Central rejected ("Bundle has content that does NOT have a .pom file"). The cause is a change in how Maven's compatibility layer installs into non-local directories. The issue is still open. A maintainer said the plugin "relies on deprecated Maven 2 features" and should be fixed, and one affected project pinned Maven 3.9.16 as a stopgap.²³

☀ Watch out for

- **"No known issues" isn't the same as no issues.** That's what the 3.10.0 release notes say, but the Central publishing bug above was reported less than two weeks after release.^{21,23}
- **Maven 4 is still a release candidate.** The download page says it's not safe for production. Nothing in this story says when GA will happen beyond "the coming weeks".^{19,16}
- **Very old plugins may break on Maven 4.** Plugins still using the long-deprecated Plexus-style injection must move to JSR-330. Maven 4 "is expected to be able to run any Maven 3 plugin that has been updated to Maven 3 best practices." Running your build on 3.9 or 3.10 with `-Dmaven.plugin.validation=verbose` shows which plugins are a risk.²⁰
- **The consumer POM doesn't apply to pom -packaged artifacts,** such as parent POMs and BOMs.²⁰

WHAT HAPPENS NEXT

The Maven team is waiting for feedback on rc-7 before calling 4.0.0 GA "in the coming weeks."¹⁶ Once that happens, Maven's support policy suggests what follows. The team maintains the last two GA series, so the arrival of a 4.0 line would eventually push 3.9 off the supported list, leaving 3.10 as the Maven 3 bridge. That's our reading of the policy, not an announced date.¹⁵ Either way, the safest path is the same: get onto 3.10 now, fix whatever it complains about, and you'll have already done much of the work Maven 4 will ask of you.

📖 Words from this story

Reactor: the part of Maven that orders and builds the projects in a multi-project build.

POM: the `pom.xml` describing a project's coordinates, dependencies, plugins and settings.

Dependencies: the libraries your project needs, identified by groupId, artifactId and version.

Transitive dependencies: the libraries your libraries depend on.

Maven Resolver: the library inside Maven that resolves dependency graphs and downloads artifacts.

BOM: a POM that pins the versions of a set of related libraries.

Multi-module build: a parent POM that lists child projects built together.

Maven Central: the main public repository of open-source Java libraries.

Classpath: the ordered list of JARs the JVM searches for classes; the first match wins.

Security hardening: a change that closes off a way an attacker could abuse existing behaviour.

Super POM: Maven's built-in default POM that every project inherits from.

Reproducible builds: builds that produce identical output from the same source.

Relocations: redirecting one artifact's coordinates to another.

Build POM: the `pom.xml` in your repository, which can use model 4.1.0 in Maven 4.

Consumer POM: the cleaned-up POM Maven 4 publishes for other tools to read.

Flatten: resolve a POM's parent and imported BOMs into one self-contained file.

Maven Wrapper: the `mvnw` script that downloads and runs a pinned Maven version.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Foojay rebuilt:** the community site moved from WordPress to Hugo, carrying over 2,250+ articles.²⁴
- **JobRunr vs Temporal:** a vendor benchmark claims 1.8 s vs 13.6 s for 1,000 three-step workflows.²⁴
- **Why each JVM warms up alone:** Azul starts a series on JIT work repeated across every replica.²⁴
- **Safer thread dumps:** tm-anon, a new MIT-licensed CLI, masks names in JVM thread dumps locally.²⁴
- **Spring AI and modular RAG:** a Spring blog post builds modular RAG pipelines with Spring AI.²⁵
- **JDD 2026:** Kraków, 20–21 October, with talks on FFM, TornadoVM's CUDA backend and more.²⁴

1. **JDK 27 Release Notes, Important Changes, and Information**, Oracle, 15 Sep 2026.
2. **Java 27 Features (with Examples)**, HappyCoders.eu.
3. **Java 27 grows up, makes better choices**, Joab Jackson, The Register, 16 Sep 2026.
4. **JEP targeted to JDK 27: 523: Make G1 the Default Garbage Collector in All Environments**, Inside Java, 26 May 2026.
5. **fix(jvm): give the heap back again on JDK 27, PR #484**, dbelokursky/tunl, GitHub, merged 28 Sep 2026.
6. **JEP 523: Make G1 the Default Garbage Collector in All Environments**, Thomas Schatzl, OpenJDK.
7. **Quality Outreach Heads-up - JDK 27: Default G1 in All Environments**, Nicolai Parlog, Inside Java, 20 Jul 2026.
8. **Containerize your Java applications for Kubernetes**, Microsoft Learn.
9. **JEP 522: G1 GC: Improve Throughput by Reducing Synchronization**, Ivan Walulya, OpenJDK (delivered in JDK 26).
10. **Compact Object Headers in Java (JEP 450 / 519 / 534)**, HappyCoders.eu.
11. **JDK 27 G1/Parallel/Serial GC changes**, Thomas Schatzl, 10 Aug 2026.
12. **Performance Improvements in JDK 27**, Per-Ake Minborg and Claes Redestad, Inside Java, 28 Sep 2026.
13. **JDK 27 Compact Object Headers and G1 Default: Measured on Spring Boot 4.1**, Ankur, ankurm.com, 24 Sep 2026.
14. **JEP 534: Compact Object Headers by Default**, Ron Veen, 29 Jun 2026.
15. **Maven Releases History**, Apache Maven (checked 10 Oct 2026).
16. **Apache Maven 4.0.0-rc-7 Release Notes**, Apache Maven.
17. **Java News Roundup: TornadoVM 7.0, Groovy 6.0, GraalVM, Hibernate, Quarkus, Gradle, Maven**, InfoQ, 28 Sep 2026.
18. **Release 3.10.0**, apache/maven, GitHub, 1 Oct 2026.
19. **Download Apache Maven**, Apache Maven (checked 10 Oct 2026).
20. **What's new in Maven 4?**, Apache Maven (checked 10 Oct 2026).
21. **Maven 3.10.0 Release Notes**, Apache Maven.
22. **Maven 3.10.0 upgrade campaign, issue #5314**, jenkins-infra/helpdesk, opened 2 Oct 2026.
23. **Issue #13388: LegacyLocalRepositoryManager.overlay() writes local-repository files into the target**, apache/maven, opened 8 Oct 2026.
24. **The New Foojay, and Who to See at JDD 2026 - JVM Weekly vol. 195**, Artur Skowronski, 8 Oct 2026.
25. **This Week in Spring - October 6th, 2026**, Spring blog, 6 Oct 2026.